

Analizador Lógico

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen - En esta práctica se estudia el funcionamiento de la comunicación serial RS232 mediante el uso de una Raspberry Pi Pico 2W, MicroPython y un osciloscopio digital. Se analiza la relación existente entre la tasa de transmisión, expresada en baudios, y el tiempo de duración de cada bit, verificando experimentalmente que el tiempo de bit corresponde al inverso de la tasa de transmisión. Además, se estudia la estructura de las tramas seriales, identificando el bit de inicio, los bits de datos, el bit de paridad y los bits de parada.

A partir de diferentes configuraciones de velocidad, número de bits de datos y paridad, se observan las variaciones producidas en las señales transmitidas. También se mide el tiempo total necesario para transmitir caracteres y tramas completas, comparando los valores teóricos con las mediciones obtenidas mediante el osciloscopio. Finalmente, se plantea la interconexión de dos Raspberry Pi Pico 2W para implementar transmisión y recepción de datos, incluyendo un mecanismo de confirmación mediante ACK y la transferencia de archivos.

Palabras clave - Bit de paridad, Comunicación serial, MicroPython, Protocolo RS232, Raspberry Pi Pico 2W, Tasa de baudios, Trama UART.

Abstract — This lab explores the operation of RS232 serial communication using a Raspberry Pi Pico 2W, MicroPython, and a digital oscilloscope. The relationship between the transmission rate, expressed in baud, and the bit duration is analyzed, experimentally verifying that the bit duration is the inverse of the transmission rate. The structure of serial frames is also studied, identifying the start bit, data bits, parity bit, and stop bits.

Using different configurations of speed, number of data bits, and parity, the resulting variations in the transmitted signals are observed. The total time required to transmit characters and complete frames is measured, comparing the theoretical values with the measurements obtained using the oscilloscope. Finally, the interconnection of two Raspberry Pi Pico 2Ws is proposed to implement data transmission and reception, including an acknowledgment mechanism using ACK and file transfer.

Keywords— Baud rate, Frame, Parity bit, RS232 protocol, Raspberry Pi Pico 2W, Serial communication, UART.

I. INTRODUCCIÓN

La comunicación serial es un método fundamental para el intercambio de información entre dispositivos digitales mediante la transmisión secuencial de bits. En esta práctica se analiza una comunicación UART generada por una Raspberry Pi Pico 2W y capturada mediante un analizador lógico, con el propósito de estudiar el comportamiento temporal de la señal y verificar experimentalmente la relación entre la tasa de transmisión y el tiempo de duración de cada bit.

Asimismo, se busca identificar los elementos que conforman una trama de comunicación serial, como el bit de inicio, los bits de datos, la paridad y el bit de parada. Finalmente, se evalúa el efecto de la frecuencia de muestreo sobre la resolución temporal de las mediciones, comparando los resultados experimentales con los valores teóricos para comprender las condiciones necesarias para realizar un análisis adecuado de señales UART.

II. OBJETIVOS

A. Objetivo General

Analizar y comprender el funcionamiento de una comunicación serial UART/RS232 mediante la relación entre la tasa de transmisión y el tiempo de bit, la identificación de la estructura y los niveles de la señal, el uso de analizadores lógicos y la implementación de programas en MicroPython.

B. Objetivos Específicos

- Comprobar la relación entre el tiempo de bit y la tasa de transmisión de datos.
- Analizar los niveles y características de una comunicación serial UART/RS232.
- Analizar la estructura del protocolo de comunicación serial, identificando bit de inicio, bits de datos, paridad y bit(s) de parada.
- Comprender el funcionamiento y las limitaciones de los analizadores lógicos en el análisis de protocolos.
- Desarrollar habilidades de programación empleando MicroPython.

III. MARCO TEÓRICO

A. Comunicación serial UART

Método de transmisión en el que los bits se envían secuencialmente a través de una línea. En una comunicación



UART (Universal Asynchronous Receiver-Transmitter), los dispositivos deben compartir parámetros como la velocidad de transmisión, número de bits de datos, paridad y bits de parada [1], [2].

B. Raspberry Pi Pico 2W

La Raspberry Pi Pico 2W es una placa de desarrollo basada en el microcontrolador RP2350, diseñada para aplicaciones de sistemas embebidos, automatización y comunicación inalámbrica. Cuenta con interfaces de comunicación como UART, I2C y SPI, además de conectividad Wi-Fi y Bluetooth, lo que permite su integración con diferentes sensores, módulos y dispositivos externos. Su tamaño compacto y bajo consumo la hacen adecuada para proyectos de adquisición, procesamiento y transmisión de datos. En este proyecto, la Raspberry Pi Pico 2W se emplea como plataforma de control y comunicación entre los diferentes componentes del sistema. [5]

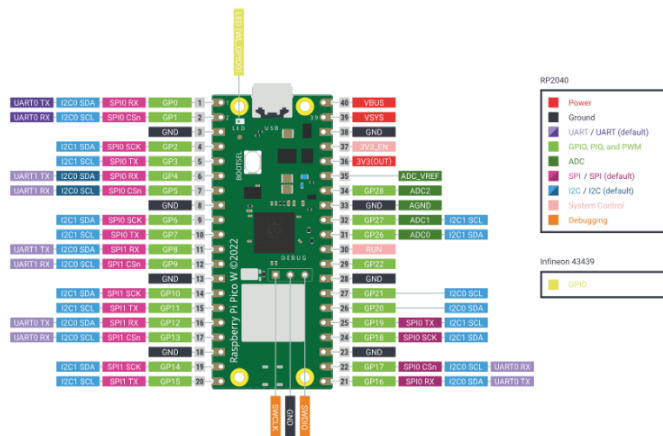


Figure 1. Raspberry Pi Pico 2W

C. Tasa de transmisión o baudios

El espectrograma representa la evolución temporal de La tasa de transmisión indica la cantidad de bits transmitidos por segundo cuando se considera una transmisión serial donde cada símbolo representa un bit. [2].

$$T_b = \frac{1}{\text{Baudios}} \text{ Ec. 1}$$

D. Trama de comunicación serial

Una transmisión serial no está formada únicamente por los bits correspondientes al carácter. [2], [3]. La información se organiza en una trama que puede contener:

1. Bit de inicio.
2. Bits de datos.
3. Bit de paridad
4. Bits de parada.

El número total de bits de una trama puede expresarse como:

$$N_{trama} = inicio + datos + Paridad (si hay) + Parada$$

Además, en una transmisión UART convencional los bits de datos se transmiten comenzando por el bit menos significativo (LSB) [2].

E. Tiempo de transmisión de un carácter

El tiempo necesario para transmitir un carácter depende de la cantidad de bits de su trama y de la duración de cada bit. [2]

$$T_{Caracter} = N_{trama} \times T_b \text{ Ec. 2}$$

F. Tiempo total de transmisión

Para transmitir varios caracteres, el tiempo total puede calcularse mediante:

El tiempo de duración de un bit depende de la tasa de transmisión y se calcula mediante [1]:

$$T_b = \frac{1}{f_s} \text{ Ec. 3}$$

G. Paridad

El bit de paridad permite detectar determinados errores en la transmisión de datos. Puede configurarse como paridad par, impar o deshabilitada. [2], [3]

Cuando se utiliza paridad, se agrega un bit adicional a la trama. Por ejemplo, con 8 bits de datos, un bit de inicio y un bit de parada:

- Sin paridad: 10 bits por trama.
- Con paridad: 11 bits por trama.

Por lo tanto, la configuración de paridad modifica tanto la estructura como el tiempo de transmisión de la trama.

H. Analizador lógico y frecuencia de muestreo

El analizador lógico permite capturar señales digitales y representar sus estados lógicos en función del tiempo. La frecuencia de muestreo f_s determina cuántas muestras se obtienen por segundo y su período de muestreo se calcula como [1]

$$T_s = \frac{1}{f_s} \text{ Ec. 4}$$

Una mayor frecuencia de muestreo produce un menor T_s , permitiendo representar con mayor resolución las transiciones de la señal.

I. Muestras por bit

La cantidad de muestras disponibles durante un bit se obtiene mediante [1]:

$$N = \frac{f_s}{f_B} \text{ Ec. 5}$$

J. Resolución temporal

Debido a que el analizador lógico toma muestras en instantes discretos, existe una incertidumbre temporal asociada al proceso de cuantización. Esta incertidumbre es aproximadamente del orden del período de muestreo [1]:

$$\Delta t = T_s$$

Por ello, aumentar la frecuencia de muestreo permite localizar con mayor precisión los flancos de la señal y medir el tiempo de bit.

IV. PROCEDIMIENTO EXPERIMENTAL

A. Configuración inicial

Para iniciar la práctica se configuró la Raspberry Pi Pico 2W para realizar una transmisión serial mediante UART. De acuerdo con la configuración establecida se seleccionó el carácter U para su transmisión a, se utilizaron 9600 baudios, 8 bits de datos, sin paridad y 1 bit de parada.

Para generar la comunicación se utilizó Micro Python mediante el siguiente código:

```
1 from machine import Pin, UART
2 import utime
3
4 uart = UART(0, baudrate=9600, bits=8, parity=None, stop=1,
5           tx=Pin(0), rx=Pin(1))
6
7 while True:
8     uart.write("U")
9     utime.sleep(1)
```

Figure 2 Código Carácter U

En esta configuración, cada carácter transmitido requiere una trama de 10 bits: un bit de inicio, ocho bits de datos y un bit de parada.

Para verificar posteriormente la duración de la señal, el tiempo teórico de cada bit se calculó mediante:

$$T_s = \frac{1}{f_s}$$

$$T_s = \frac{1}{9600} = 104,17 \mu s$$

Para la trama se calculo

$$T_{trama} = 10 \text{ bits}$$

$$T_{trama} = 10(104.17\mu s) = 1.042 \text{ ms}$$

Estos valores se utilizan como referencia para comparar las mediciones obtenidas posteriormente con el analizador lógico

B. Conexión, configuración y captura de la señal

Una vez configurada la Raspberry Pi Pico 2W, se realizó la conexión con el analizador lógico. El canal CH1 se conectó al terminal TX correspondiente al GPIO0 de la Raspberry Pi Pico 2W, mientras que la terminal GND del analizador se conectó al pin de tierra de la placa.

Posteriormente, se configuró el software Logic 2 para realizar la adquisición de la señal. Inicialmente se estableció una frecuencia de muestreo de 1 MS/s, un disparo (trigger) en Falling Edge y un tiempo de captura de 2 segundos.

Async Serial

Input Channel *	00. Channel 0
Bit Rate (Bits/s)	9600
Bits per Frame	8 Bits per Transfer (Standard)
Stop Bits	1 Stop Bit (Standard)
Parity Bit	No Parity Bit (Standard)
Significant Bit	Least Significant Bit Sent First (Standard)
Signal inversion	Non Inverted (Standard)
Mode	Normal
☒ Show in protocol results table ☒ Stream to terminal	
Learn more about setup options in our User Guide.	
Reset	Cancel Save

Figure 3. Configuración del analizador lógico en Logic 2.

V. ANÁLISIS CAPTURAS

A. Carácter U

La señal capturada permite observar la sucesión de niveles lógicos correspondientes a la transmisión UART.

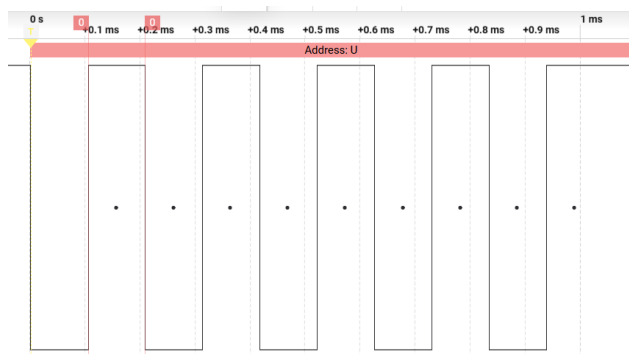


Figure 4. Trama Carácter U

Podemos observar la señal completa capturada durante los 2 s de adquisición.

A partir de la captura obtenida mediante Logic 2, se utilizaron los marcadores temporales para medir experimentalmente el tiempo de duración de la trama

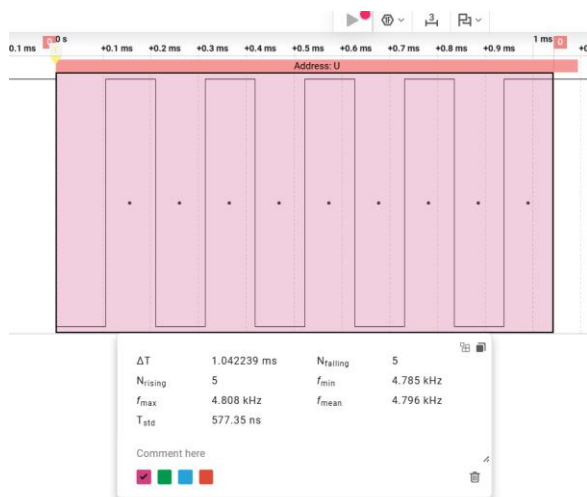


Figure 5. Trama completa Carácter U (1,0422ms)

Después del Stop bit la línea UART queda normalmente en estado alto (1), por lo que el hecho de que la señal permanezca en alto más allá de 1.042 ms no significa que la trama dure más

La medición del intervalo entre transiciones permite determinar experimentalmente el tiempo de duración de un bit y compararlo con el valor teórico.

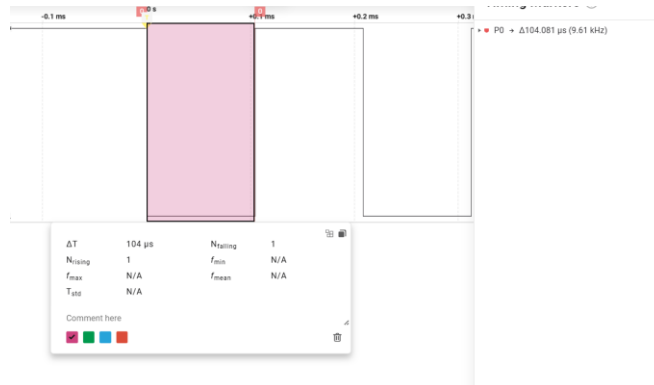


Figure 6. Tiempo de duración Bit (140us)

Una vez identificado los datos se procedió a comparar los resultados mediante el calculo del porcentaje de error:

$$\%e(Bit) = \frac{|104.22-104.166|}{104.166} \times 100 = 0.152\%$$

$$\%e(Trama) = \frac{|1.0422-1.042|}{1.042} \times 100 = 0.0191\%$$

Tabla 1. Resultados

Teorico	Experimental	%e
104.166 us	104 us	0.152
1.042 ms	1.04222 ms	0.0191

B. Medición carácter nuevo (^)

Para el siguiente paso se mantuvo la configuración presente y se introdujo a analizar un nuevo carácter (^)

```
1 from machine import Pin , UART
2 import utime
3
4 uart = UART (0, baudrate =9600 , bits =8, parity =None , stop =1,
5 tx=Pin (0) , rx=Pin (1) )
6
7 while True :
8     uart . write ("^")
9     utime . sleep (1)
```

Figure 7. Código Carácter (^)

Este carácter mantiene la misma cantidad de bits tiene una trama de 10 bits: un bit de inicio, ocho bits de datos y un bit de parada. Y se utiliza el mismo tiempo de baudios por lo que se utiliza los valores teóricos cuya calculados para compararlos posteriormente

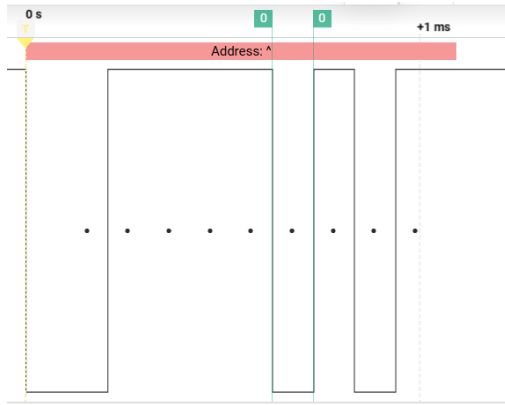


Figure 8. Trama Carácter (^)

Podemos observar la captura completa de los 2 s de adquisición para el carácter (^), donde nuevamente se observa un único pulso de trama seguido de un largo período en alto, replicando el mismo patrón temporal registrado para el carácter U y confirmando que el intervalo entre envíos no depende del carácter transmitido.



Figure 9. Trama completa Carácter (^) (1,0429ms)

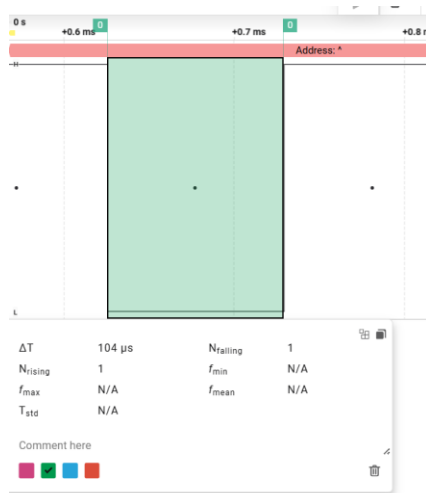


Figure 10. Tiempo de bit (^) (104us)

Para el carácter (^) el analizador lógico midió una trama completa de 1.042936 ms y un tiempo de bit de 104 μs. Comparando con los valores teóricos calculados para 9600 baudios (T_b = 104.166 μs; T_{trama} = 1.0417 ms), el error porcentual obtenido es:

$$\%e(bit) = \frac{|104 - 104.166|}{104.166} \times 100 = 0.159\%$$

$$\%e(trama) = \frac{|1.04293 - 1.042|}{1.042} \times 100 = 0.089\%$$

Tabla 2. Resultados

Teorico	Experimental	%e
104.166 us	104 us	0.159
1.042 ms	1.04222 ms	0.089

Los errores obtenidos son consistentes con los del carácter U (0.152 %), confirmando que, para una misma configuración de baudios y trama (10 bits, sin paridad), el tiempo de bit medido es independiente del carácter transmitido, ya que depende únicamente de la tasa de transmisión configurada y no del valor lógico de los datos.

C. Medición carácter repetido (^)

Posteriormente se realizó el envío del carácter (^) 5 veces para analizarlo

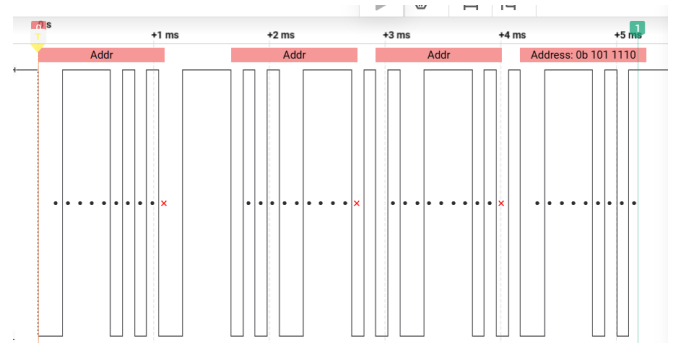


Figure 11 Trama Carácter (^) Envío 5 Veces

Podemos observar las 5 tramas consecutivas del carácter (^), cada una identificada por Logic 2 con su dirección decodificada ("Addr" / "0b 101 1110"), separadas por intervalos de reposo en alto.

Se aprecia que las tramas conservan la misma forma de onda entre sí, lo que sugiere una temporización consistente en el envío sucesivo de caracteres desde la Raspberry Pi Pico 2W



Figure 12 Análisis tiempo total de la Trama Carácter (^)

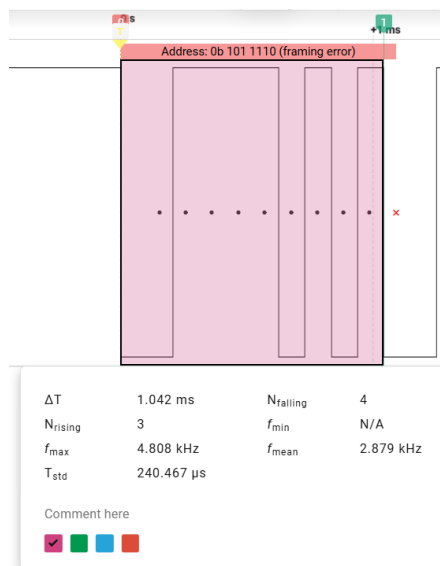


Figure 13 Análisis de una Trama Carácter (^)



Figure 14 Análisis tiempo de bit Carácter (^)

Para el carácter (^) el analizador lógico obtuvimos los mismos resultados vistos anteriormente cuando enviamos solo una trama.

El envío consecutivo del carácter (^) 5 veces arrojó un tiempo total de trama de 5.201459 ms. El valor teórico para 5 caracteres de 10 bits a 9600 baudios es

$$T_{total} = 5 \times 1.0417 \text{ ms} = 5.2083 \text{ ms}$$

con un error de:

$$\%e = \frac{|5.201459 - 5.2083|}{5.2083} \times 100 = 0.0191\%$$

D. MENSAJE

Finalmente se procedió a enviar el mensaje asignado de la UMNG a la misma configuración para analizarla. Para del procedimiento se transmitió la cadena:

“UMNG_2026_LIDER_EN_TELECOMUNICACIONES”

A partir de esta transmisión se analizó la duración de la secuencia completa y la estructura de las tramas correspondientes a los diferentes caracteres

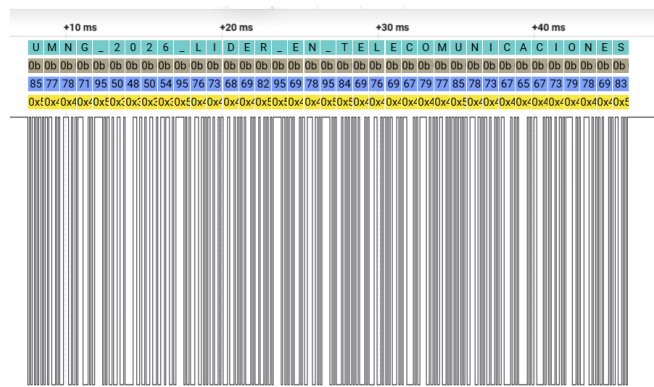


Figure 15 Trama mensaje de la UMNG

Podemos observar la decodificación completa del mensaje "UMNG_2026_LIDER_EN_TELECOMUNICACIONES", en la que Logic 2 identifica correctamente cada uno de los 37 caracteres en sus representaciones ASCII, binaria, decimal y hexadecimal.

Esto confirma que, con la configuración de 9600 baudios, 8N1 y una frecuencia de muestreo de 1 MS/s, el analizador decodifica de forma confiable tramas largas y consecutivas

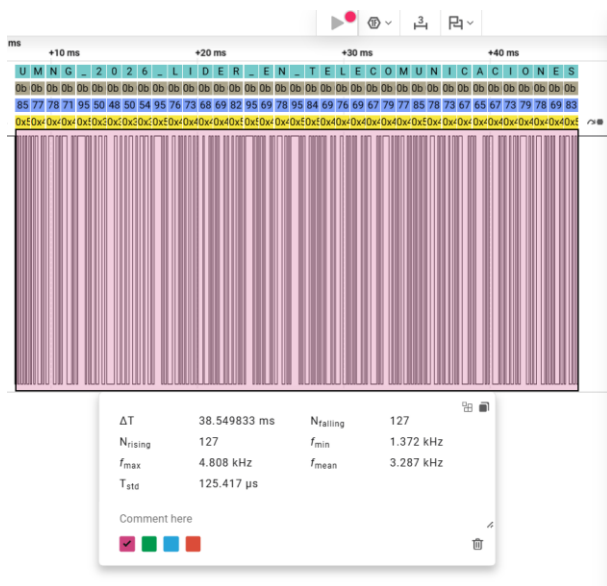


Figure 16 Tiempo total Trama mensaje de la UMNG

El mensaje tardó 38.549833 ms en transmitirse. El valor teórico, con trama de 10 bits (8N1) a 9600 baudios, es:

$$T_{total} = 37 \times 10 \times 104.166 \mu s$$

$$T_{total} = 38.542 \text{ ms}$$

$$\%E = \frac{|38.549833 - 38.542|}{38.542} \times 100 = 0.020\%$$

El error inferior al 0.03 % confirma que, incluso para tramas largas encadenadas, la desviación entre el tiempo teórico y el medido se mantiene mínima cuando se emplea una frecuencia de muestreo de 1 MS/s, muy por encima del criterio práctico de 10 muestras/bit. A diferencia de la captura del carácter (^) repetido.

VI. MENSAJE CAMBIANDO CONFIGURACIÓN

A. MENSAJE DE LA UMNG FRECUENCIA DE MUESTREO

Finalmente, se evaluó experimentalmente el efecto de modificar la frecuencia de muestreo del analizador lógico. Para ello se mantuvieron las condiciones de transmisión y se realizaron capturas independientes utilizando diferentes valores de fs.

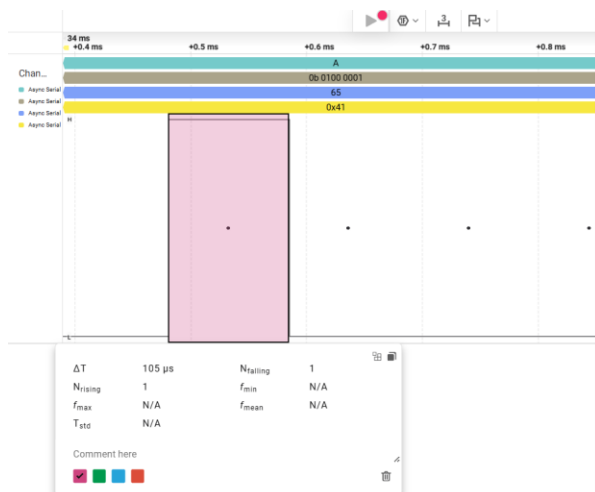


Figure 17. Mensaje Modificada

Podemos observar muestra el carácter 'A' (0x41) decodificado donde el tiempo de bit medido en este punto es de 105 μs, muy cercano al valor teórico de 104.17 μs (error de 0.8 %), lo que confirma que las cuatro vistas de decodificación son consistentes entre sí y con la temporización esperada.

Posteriormente, para cada combinación de frecuencia de muestreo y tasa de transmisión se determinará el número de muestras disponibles por bit para diferentes configuraciones a través de la ecuación 5.

Para cada caso se mostrará la primera configuración de 25k/s.

A 1200 Baudios

$$N = \frac{25 \text{ k}}{1200} = 20,83$$

A 9600 Baudios

$$N = \frac{25 \text{ k}}{9600} = 2,6042$$

A 57600 Baudios

$$N = \frac{25 \text{ k}}{57600} = 0,434$$

A 115200 Baudios

$$N = \frac{25 \text{ k}}{115200} = 0,217$$

Para ver la tabla completa dirige al **ANEXO A**

A 1200 Baudios

Baudios /fs	25k	50k	100k
1200	20,833333	41,666667	83,333333
%>10	✓	✓	✓
Tiempo bit (us)	840	820	830

A 9600 Baudios

Baudios /fs	25k	50k	100k
9600	2,6041667	5,2083333	10,416667
%>10	X	X	✓
Tiempo bit (us)	80	100	100

A 57600 Baudios

Baudios /fs	25k	50k	100k
57600	0,4340278	0,8680556	1,7361111
%>10	X	X	X
Tiempo bit (us)	80	20	10

A 115200 Baudios

Baudios /fs	25k	50k	100k
115200	0,2170139	0,4340278	0,8680556
%>10	X	X	X
Tiempo bit (us)	40	20	10

se observa el comportamiento esperado: a 1200 baudios el criterio de ≥ 10 muestras/bit se cumple incluso a 25 kS/s, mientras que a 9600 baudios se requiere al menos 100 kS/s, y a 57600 y 115200 baudios se necesitan 1 MS/s y 2 MS/s respectivamente.

El tiempo de bit medido se aproxima al valor teórico calculado (Ver Ecuación 1) a medida que aumenta fs, y se aleja notablemente cuando el criterio no se cumple; por ejemplo, a 9600 baudios con fs=25 kS/s (2.60 muestras/bit) se midió un tiempo de bit de 80 μ s frente a un valor teórico de 104.17 μ s, un error superior al 23%.

VII. ANÁLISIS Y DISCUSIÓN DE RESULTADOS

- Tiempo de bit y tasa de transmisión

Los resultados obtenidos para el carácter U, el carácter (^) y el mensaje completo confirman experimentalmente que $T_b = 1/\text{Baudios}$, con errores porcentuales entre 0.02 % y 0.16 % cuando la captura se realiza con una frecuencia de muestreo de 1 MS/s (~104 muestras/bit a 9600 baudios), muy por encima del criterio práctico de 10 muestras/bit. Esto valida el modelo teórico de la trama UART (bit de inicio + 8 bits de datos + bit de parada = 10 bits) usado para calcular Ttrama y Ttotal.

- Efecto de la frecuencia de muestreo

Al reducir fs, aumenta el período de muestreo $T_s = 1/fs$ y, con ello, la incertidumbre temporal en la localización de los flancos. La Tabla 1 muestra que este efecto es más crítico a medida que aumenta la tasa de baudios: a igual fs, una trama a 115200

baudios dispone de muchas menos muestras por bit que una trama a 1200 baudios, por lo que requiere frecuencias de muestreo proporcionalmente mayores para cumplir el criterio de ≥ 10 muestras/bit. Esto es consistente con la relación $N = fs/fB$ descrita en la guía.

Muestreo suficiente no garantiza una decodificación correcta

Un hallazgo relevante de la práctica fue el error de framing detectado en una de las tramas del carácter (^) enviado repetidamente (Figura 11), a pesar de que el tiempo de trama medido (1.042 ms) coincidía con el valor teórico y de que la frecuencia de muestreo (1 MS/s) cumplía ampliamente el criterio de 10 muestras/bit. Esto evidencia que el criterio de muestras por bit es necesario, pero no suficiente, para garantizar una decodificación confiable: los parámetros del protocolo configurados en el analizador (baudios, bits de datos, paridad, bits de parada) deben coincidir exactamente con los del transmisor.

VIII. PREGUNTAS DE ANÁLISIS DE LA GUÍA

A. ¿Qué efecto tiene el cambio en la tasa de muestreo para determinar el tiempo de bit?

Al disminuir fs, el período de muestreo T_s aumenta y con él la incertidumbre en la localización de los flancos de la señal, lo que provoca que el tiempo de bit medido se aleje del valor teórico. Esto se observó experimentalmente a 9600 baudios con fs=25 kS/s, donde el tiempo de bit medido (80 μ s) difiere en más de 23 % del valor teórico (104.17 μ s), mientras que con fs=1 MS/s el error se reduce a menos de 0.2 %. Aumentar fs reduce T_s y permite ubicar los flancos con mayor precisión, acercando la medición al valor teórico.

B. ¿Qué desventaja operativa o de almacenamiento puede presentarse al configurar tasas de muestreo excesivamente altas (24 MS/s) durante capturas de larga duración?

Frecuencias de muestreo muy altas generan un volumen de datos considerablemente mayor por segundo de captura, lo que incrementa el uso de memoria RAM y de almacenamiento en disco, así como el tiempo requerido para procesar, exportar o transferir la captura (Logic 2 incluso limita el tamaño del buffer de memoria configurable).

Además, dado que el criterio práctico de 10 muestras/bit ya se satisface con frecuencias muy inferiores a 24 MS/s para las tasas de baudios usadas en este laboratorio, el uso de fs excesivamente altas no aporta una mejora proporcional en la precisión de la decodificación y resulta ineficiente en términos de recursos.

IX. CONCLUSIONES

- Se comprobó experimentalmente que el tiempo de duración de cada bit presentó una relación inversa con la tasa de transmisión configurada, obteniéndose mediciones cercanas a los valores teóricos cuando se emplearon condiciones adecuadas de captura.

- Se analizaron los niveles lógicos y el comportamiento temporal de la señal serial, permitiendo observar el estado de reposo de la

línea, las transiciones generadas durante la transmisión y la variación de la señal de acuerdo con la información enviada.

- Se identificó la organización de las tramas transmitidas, reconociendo la función del bit de inicio, los bits correspondientes a los datos, la configuración de paridad cuando esta se encuentra presente y los bits de parada, elementos que determinan tanto la estructura como la duración de la transmisión.

- Se verificó que la frecuencia de muestreo del analizador lógico influye directamente en la precisión de las mediciones, debido a que una cantidad insuficiente de muestras por bit aumenta la

incertidumbre temporal y puede producir diferencias significativas frente a los valores esperados.

- Se evidenció que una frecuencia de muestreo elevada no garantiza por sí sola una decodificación correcta, ya que también es necesario que parámetros como los baudios, el número de bits de datos, la paridad y los bits de parada estén correctamente configurados y coincidan con los utilizados por el transmisor.

- Se desarrollaron habilidades prácticas de programación mediante MicroPython y la configuración de la Raspberry Pi Pico 2W, permitiendo generar y transmitir información a través de UART para posteriormente capturar, visualizar y analizar experimentalmente las señales obtenidas.

ANEXO A

Baudios /fs	25k	50k	100k	200k	250k	500 K	1M	2M	4M	8M	12M	16M	24M
1200	20,833	41,667	83,333	166,667	208,333	416,667	833,333	1666,667	3333,333	6666,667	10000	13333,333	20000
%>10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Tiempo bit (us)	840	820	830	835	832	834	833	833,5	833,25	833,375	833,417	833,313	833,375

Baudios /fs	25k	50k	100k	200k	250k	500 K	1M	2M	4M	8M	12M	16M	24M
9600	2,604	5,208	10,417	20,833	26,0416	52,083	104,167	208,333	416,667	833,333	1250	1666,667	2500
%>10	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Tiempo bit (us)	80	100	100	100	108	104	105	104	104,25	104,125	104,167	104,188	104,167

Baudios /fs	25k	50k	100k	200k	250k	500 K	1M	2M	4M	8M	12M	16M	24M
57600	0,434	0,868	1,736	3,472	4,340	8,681	17,361	34,722	69,444	138,889	208,333	277,778	416,667
%>10	X	X	X	X	X	X	✓	✓	✓	✓	✓	✓	✓
Tiempo bit (us)	80	20	10	15	20	18	18	17,5	17,5	17,375	17,333	17,375	17,33

Baudios /fs	25k	50k	100k	200k	250k	500 K	1M	2M	4M	8M	12M	16M	24M
115200	0,217	0,434	0,868	1,736	2,170	4,340	8,681	17,361	34,722	69,444	104,167	138,889	208,333
%>10	X	X	X	X	X	X	X	✓	✓	✓	✓	✓	✓
Tiempo bit (us)	40	20	10	10	8	8	8	8,5	8,75	8,75	8,667	8,688	8,667